



IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Program
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Lecture 1
Introduction

Welcome to the course programming concepts with java. So, as the title suggests we are going to be talking about programming concepts in general as well as a language java in particular. So, let us get started with understanding what a programming language is.

(Refer Slide Time: 00:30)



So, a language in normal terms is a way with which we communicate. So, when we have to tell somebody about something that we want to convey to them an idea or a concept we use a natural language. In the same way we use a programming language to convey computational ideas to a machine that can execute them. So, we want to express some instructions to be executed by some machine and for this we use a programming language.

Now traditionally programming languages were fairly closely tied traditionally being very early days to the way that programming computers were built. So, if you look at a computer even today normally it has an arithmetic and logic unit or sometimes called a central processing unit and it will have a memory. And the memory stores data but a

memory is not a place in which you can perform operations you cannot take one memory location the value there a value at another memory location for example and add them.

So, that has to be done inside the ALU or the cpu. So, when you write something to be computed you have to first bring the values that are in memory into your ALU or CPU. So, there you have these special locations called registers. So, you have a small number of places where you can store values these registers now can be operated on. So, you can say perform an operation on these registers and then finally you have to take these values and put them back.

So, that is essentially how at a low level computation proceeds in a traditional architecture and the old programming languages used to actually say this. So, you had instructions of this kind take some memory location and transport its value to a register R. Do this for as many memory locations as you need of course there are only a small number of registers. So, you cannot copy everything at one time.

So, you copy a few values operate on them. So, take R 1 and R 2 say add it and put it back in R 1 and then you would take the contents of a register and put it back in memory. So, that you can free up the register for later work right and also to persist the storage if you do not put it back in the memory the value is lost when the computer is switched off. So, this is obviously a very tedious and error prone way to program because not only does it take you many steps to express a basic thing like assign x the value $y + z$.

Because you have to bring all these values y into the register z into the register add then put it back in x and so on. But also you could easily miss a step or two or make a mistake about which memory location you are talking about. So, this low level programming is not what we want.

(Refer Slide Time: 03:03)



IIT Madras
BSc Degree

Abstraction

- Abstractions used in computational thinking
 - Assigning values to named variables
 - Conditional execution
 - Iteration
 - Functions / procedures, recursion
 - Aggregate data structures — arrays, lists, dictionaries
- Express such ideas in the programming language
 - Translate "high level" programming language to "low level" machine language
 - Compilers, interpreters
- Trade off expressiveness for efficiency
 - Less control over how code is mapped to the architecture
 - But fewer errors due to mismatch between intent and implementation



So, what we want to do actually is to match the level of abstraction that we have when we do computational thinking. So, as we know when we do computational thinking one of the primary things that we need is a way to remember values. So, we have names or variables which we associate with values and we have ways of assigning these values to these names. So, we have an assignment statement take the name x and assign it the value 5.

Then we have not just a sequence of assignments but conditional flow. So, we say if the value of x is positive do something otherwise do something else. So, we can have conditional execution. So, we have control flow we can have repeated execution right for every x ranging from 1 to 10 do something or we could have repeated execution till a condition holds while x is less than 10 do something.

Then we group together logical units into functions supposing we want to compute repeatedly say the square of a number then so, writing x times x all over the place we might write a function square which takes x returns x squared and of course we can do more complicated things in that something like factorial or something which is not easy to compute in an inline expression.

And these functions and procedures can also call themselves. So, we can express inductive computations where a value of a function can be expressed in terms of smaller values and finally we have aggregate data structures. So, we have these composite data

structures like lists and arrays and dictionaries which allow us to group together a number of values together into a logical unit which we can then operate on.

So, over time programming languages evolve to embrace these features not all in one shot but gradually. So, the modern languages that we have today can express all these things for example the language python which we are familiar with. So, of course there is now a gap because we have our programming language and then we have the machine and there is no way that our programming language can directly be understood by the machine because now it is talking in terms of these abstract concepts like variables and while loops and for loops.

So, what we have instead is this compiler. So, a compiler sits in between our programming language and the machine and it translates our programming language into instructions that the machine will understand. Now a compiler could also be replaced by something called an interpreter. So, the difference between a compiler and an interpreter is basically whether or not it produces a machine language program in one shot or it does it instruction by instruction.

So, do we read our high level program one line at a time and execute it or do we translate the whole thing into a low level program and run it that is what a compiler does whereas an interpreter does it one sentence at a time so to speak. So, as in every such situation what we have gained is expressiveness by moving to a higher level of communication. So, we are talking in terms of variables loops recursion and so on.

But these are not now in our control in terms of how they are implemented on the machine. So, the compiler is in control of that. So, if we want some fine grained control for instance we want to use the minimum amount of memory of the fewest registers possible in order to achieve a particular computation this is no longer in our control. So, there is an efficiency tradeoff. So, we could lose efficiency because the mapping of our high level program to the machine architecture is done automatically by a compiler.

But on the other hand what we gain definitely is that we are able to reason about our program at the level at which we designed it. So, the code matches our intent more clearly and therefore logical errors which are more the problem than anything else are

easier to detect and avoid. So, we gain in expressiveness in terms of being able to write code and read code and maintain code more easily even though there is a small loss in efficiency.

And the small loss in efficiency is really not that relevant anymore because we know that computers have got more resources than many more. I mean even our phones have much more resources than a computer did say 20 years ago. So, we really do not have to worry too much about this very micro optimization that we could do if we wrote a code by hand in a machine language.

(Refer Slide Time: 07:21)



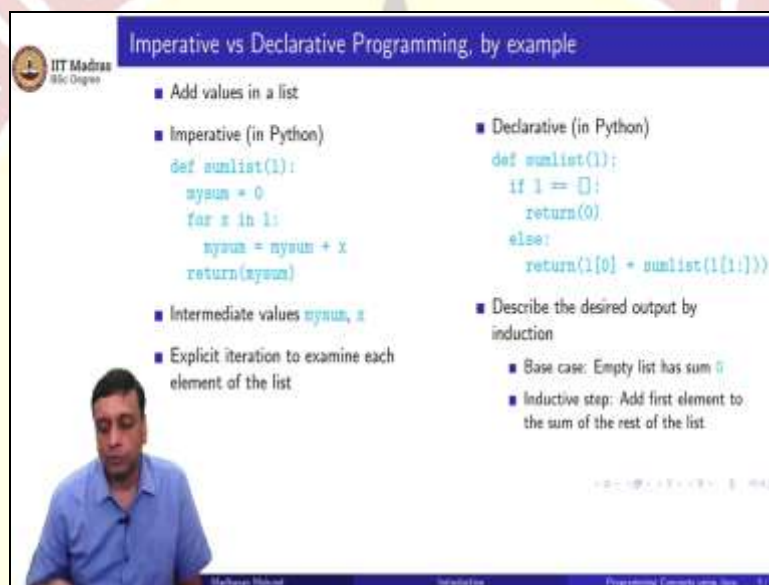
So, different programming languages have different styles. So, there are two dominant trends in programming one is called imperative and the other is called declarative. So, imperative programming is the one that we are more familiar with both through our computational thinking and through our python experience. So, in an imperative program we explicitly give instructions on how the values that we want to achieve are computed.

So, we define intermediate variables we give step by step instructions we have loops we go through lists from beginning to end and so on. On the other hand in a declarative program right what you typically give is an abstract description of how the output depends on the input. So, this is very often suitable if you have an inductive kind of definition as we will see.

So, you can exploit the inductive structure and express say for example factorial of n in terms of factorial of $(n - 1)$ which is a smaller quantity to compute multiplied by the current value n . So, how to combine the current argument with smaller computations to get the result and one advantage of this is that very often you can avoid using intermediate variables and you can define your function in terms of smaller functions and smaller transformations.

So, it is easier to explain why your function is doing what it is doing rather than through a sequence of explicit manipulations of values and variables.

(Refer Slide Time: 08:46)



Imperative vs Declarative Programming, by example

- Add values in a list
- Imperative (in Python)


```
def sumlist(l):
    mysum = 0
    for x in l:
        mysum = mysum + x
    return(mysum)
```

 - Intermediate values `mysum, x`
 - Explicit iteration to examine each element of the list
- Declarative (in Python)


```
def sumlist(l):
    if l == []:
        return(0)
    else:
        return(l[0] + sumlist(l[1:]))
```

 - Describe the desired output by induction
 - Base case: Empty list has sum 0
 - Inductive step: Add first element to the sum of the rest of the list

So, rather than get into some very complicated theoretical definition of what these two things mean. Let us just look at a couple of examples in a language we are familiar with namely python. So, let us look at a very simple example first how to add all the values in the list of numbers. So, if you want to add all the values in list of numbers here is an imperative program.

So, what it does is it takes a list l right and it starts by initializing a sum variable to 0 and then you have a loop which iterates over all the elements in l . So, for every x and l you add x to the current value of my sum and update it and finally you return this. So, this is a very explicit step by step way to iterate through all the elements in a list and add them up and return the sum.

If you do it declaratively as we said typically you will exploit some kind of inductive structure. So, the inductive structure of this function is something that you have to think about of course but once you get it, it is very easy to explain why it is correct without explaining the mechanics of how the computation is done. So, you can say without any hesitation that the sum of an empty list is 0 because there are no values.

On the other hand if you have a non empty list then you can take the list right and then you can separate it out into the first value and the rest and if I can add up this rest then the sum of the whole list is just x plus the sum of the rest. So, that is what this python program here is saying is saying that if the list is empty then return zero otherwise take the first element and add it to the inductive sum of the slice starting at the next position. So, `l[1:]` is the rest of the list.

So, what have we achieved well. We have avoided these intermediate values `my sum` and `x` which are there in our imperative program. So, that is one thing and the imperative program also has to explicitly explain how to go through this list. So, we have to kind of understand the list structure and know that we go through from beginning to end and so on. Whereas in the declarative style you just specify the output.

So, it is kind of it should be at least evident by inspecting the structure of the program that what you are computing is the right value. So, for instance here you might have to be a little careful in the imperative thing that you initialize `my sum` correctly that you have not missed any element in the list and so on whereas here it is much more transparent. So, this is one example.

(Refer Slide Time: 11:11)

Imperative vs Declarative Programming, by example, ...

- Sum of squares of even numbers upto n
- Imperative (in Python)


```
def sumsquareeven(n):
    mysun = 0
    for x in range(n+1):
        if x%2 == 0:
            mysun = mysun + x*x
    return(mysun)
```
- Declarative (in Python)


```
def even(x):
    return(x%2 == 0)

def square(x):
    return(x*x)

def sumsquareeven(n):
    return(
        sum(map(square,
                  filter(even,
                        range(n+1)))))
```
- Can code functionally in an imperative language!
- Helps identify natural units of (reusable) code

Madhavan Mukund

So, let us look at a slightly more complicated example. So, supposing we want to add up all the square numbers from 0 to n . So, we want 0 squared + 2 squared + 4 squared etcetera up to n squared if n is even or $n - 1$ squared if n is odd. So, here is a again a simple imperative program to do it in python right you take n as your input then as before you initialize a sum to be 0 you run through every x from zero to n and remember that range in python goes to one less than the output.

So, if I say range n I will only go up to n minus one. So, I have registered range $n + 1$. So, going from zero to n you take each x examine whether or not it is even by dividing by two and checking if the remainder is zero and if it is even by that check then you will take x times x which is x squared and add it to my sum. So, this is very explicitly saying each number check if it is even if it is even square it add it to the sum nothing wrong with this but this is a very direct imperative style of defining this.

So, let us look at how a declarative version of this might look. So, the first thing is that we want to take these magic expressions right and identify that they have some meaning. So, we pull out these functions even x and square x . So, even x is what is called a predicate. So, a predicate is a function which returns true or false. So, this tells you whether x is even or x is odd.

If it is true x is even if it is not true x is for whereas square is a conventional function it takes a number and produces a different number in this case it produces the square of that number. So, now let us look at how we would process this. So, what we want to do

is take our list 0 to n right we want to take out the even numbers from it. So, now we have even numbers from 0 to n then we want to square each element and then finally we want to sum it.

So, this is a logical progression of our computation. We first take our numbers remove all the odd numbers keep only the even ones then each even number has to be squared and finally we have to add it up. So, that is what this code is doing now. So, you start with this range as before these are the number 0 to n and now you apply this function filter to it. So, filter takes a predicate and retains all those numbers in the list which match that predicate.

So, this does the first transformation this one. Now I want to do a second transformation that is achieved by map. So, map takes this function and applies it to every element of the list it is provided with in this case the list it is provided with is all the even numbers that survive the filter. And finally I have used here python's built in function sum which takes a sequence and adds it up.

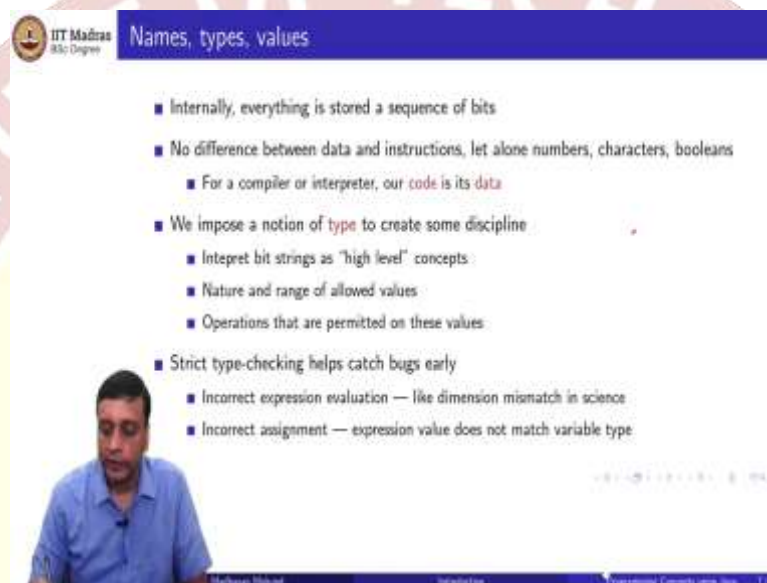
So, notice that we have described this pipeline without having to store any intermediate values anywhere and we have also broken it up into the logical steps which are implied by the description of the problem take the list of numbers filter out the even ones square them and then add them. So, this tells us that using a declarative or a functional style is not a property of the language.

It is not that if you use python or if you use some other language like python you must program imperatively. It says that it is a question of how you break up the problem and how you think about it. So, here we have the same problem described in two very different ways. So, why would you use a declarative style? Well the declarative style as we saw first of all avoids these magic variables.

I do not have my $\sum x_i$ I do not worry about if I am reading the code what are these values denoting right its very transparent what is going on. The second thing is that we have these natural units of code. So, we have this even and we have the square which are embedded as expressions in my imperative style. Now of course even in the imperative style I could pull them out as functions.

But the point is that the functional style or the declarative style encourages you to do that because there is no real way to put that into this without calling it a function. So, you can take a functional or declarative style and make your python or java as we will see code more readable. So, this is something that one should always think about how to break up. So, avoid these large blocks of instructions right and break them up into smaller meaningful units of code which are easier to process.

(Refer Slide Time: 15:41)



IIT Madras
800 Degrees

Names, types, values

- Internally, everything is stored a sequence of bits
- No difference between data and instructions, let alone numbers, characters, booleans
 - For a compiler or interpreter, our **code is its data**
- We impose a notion of **type** to create some discipline
 - Interpret bit strings as "high level" concepts
 - Nature and range of allowed values
 - Operations that are permitted on these values
- Strict type-checking helps catch bugs early
 - Incorrect expression evaluation — like dimension mismatch in science
 - Incorrect assignment — expression value does not match variable type

Madhavan Muralidharan

So, that is one part of our programming language which is how to structure the instructions but finally the instructions manipulate values and these values have to be stored and as we all know in a computer everything is finally a sequence of zeros and ones. So, there is really no distinction between integers and floats and characters at the level of the raw storage.

There is even not a difference between programs and data. So, for example if you take a compiler a compiler reads what we think of as a program and transforms it into another program. So, for a compiler the program that we write is actually data. So, in fact it is all sitting in the same memory. So, there is really no a priori way of identifying what is data what is program in data what is integer what is float what is good.

So, what we need to do is we need to impose some type discipline we need to say okay treat this part of memory as an integer treat that part of memory as a float create. This other part as a character. So, this allows us to interpret these different sequences of bits

or bit strings as higher level concepts that we can manipulate. We can also associate along with that some more constraints we can say that this should be a number between this should be a positive number.

Or I mean it depends on the programming language how much power it gives you but we could even restrict the not just the nature is it a integer is it a float and all that but even the range we can say that this is a positive number for instance. And finally of course once we have brought it into this mode where we know what type of value we are talking about we have a range of operations which are allowed on them.

So, if they are numbers we have some arithmetic operations if they are Booleans we have some logical operations if they are characters we can concatenate and so on. So, this is what we use to make sense out of the data. So, that is the first level at which we need to use types which is just to take a kind of undistinguished sequence of zeros and ones and make sense out of it.

But we are also going to be interested in other aspects of this. So, one aspect of this is that if we do this it also helps us to find errors. So, if we take something which is of two incompatible types and we try to apply an operation then we can tell that we have done something wrong. So, if we make mistakes having types helps us find it and helps us find it early. So, we could have an incorrect assignment or an expression which has a mixed type input which does not work. So, if you take in python you try to take a list right and you try to add a number it will say this is not correct.

(Refer Slide Time: 18:19)



- Collections are important
 - Arrays, lists, dictionaries
- Abstract data types
 - Structured collection with fixed interface
 - Stack is a sequence, but only allows `push` and `pop`
 - Separate implementation from interface
 - Priority queue allows `insert` and `delete-min`
 - Can implement a priority queue using sorted or unsorted lists, or using a heap
- Object-oriented programming
 - Focus on data types
 - Functions are invoked through the object rather than passing data to the functions
 - In Python, `mylist.sort()` vs `sorted(mylist)`



Now one of the things that we will be focusing on in this course is something called object oriented program which comes out of this notion of an abstract data type. So, remember that in addition to the simple types of intern float and that what is really important for us when we write programs is to be able to collect them in groups. So, most programming languages will give us these basic groups like arrays or lists or in python we also have dictionaries which are essentially like list but indexed by keys rather than by positions but in a more general setting we need to create data types to suit our requirements.

So, we can already create more complicated types like stacks for instance. So, a stack is a sequence but unlike a normal sequence or an array or a list which I can manipulate arbitrarily in a stack I can only work at one end. So, I can allow push and pop so I should think of a stack really as a vertical thing okay. So, I push and I pop. So, I am not allowed to directly go and look inside this thing. So, this is not allowed.

So, it is a sequence indeed and internally it is represented as a sequence and there is nothing that prevents me from physically doing it but logically a stack only allows push and pop. So, this is what is called an interface. So, we have some collection of values which are there in that data type and we have an interface which tells us which buttons we can press what functions we can operate on this.

So, one of the nice things about doing it this way is that you can separate the interface from the implementation. So, how these buttons work and what they do. So, this is again

a question of declarative versus imperative in a different style. So, when we say that push tells us the last value that was put into the stack I mean push puts a value into the stack and remembers the last value and pop remembers the order and brings out the last value right how do we know which is first and which is last.

So, think of a priority queue remember a priority queue something in which you can insert elements but each element has an associated priority or level. So, the things come out not in the order in which they went in. So, it is not like a queue where you are waiting to buy a ticket where the person who entered last leaves last but it is rather based on some status of the person.

So, depending on their status the next highest person in the queue whose waiting gets served next regardless of when that person arrived. So, this is a priority queue. So, we could implement such a queue in many different ways we can maintain this elements as a sequence can maintain it in sorted order unsorted order and we can depending on how we have done it we have to scan and find the highest priority element and bring it out.

Or we could be clever and use this clever tree structure called a binary heap and do these things very efficiently but at the level of insert and delete max which are our two interface operations on a priority queue all of these implementation should give us the same answer. So, in some sense we can optimize the implementation without affecting the interface. So, this is one of the properties of an abstract data type that if your data type is being used in some your own code or by somebody else's code.

If you make changes to the implementation it should not affect their code so the separation of concerns that comes. So, object oriented programming takes this notion of abstract data type as its central theme. So, it organizes all of its work around abstract data type. So, normally when we write programs or we start learning how to write programs we are always focusing on control flow.

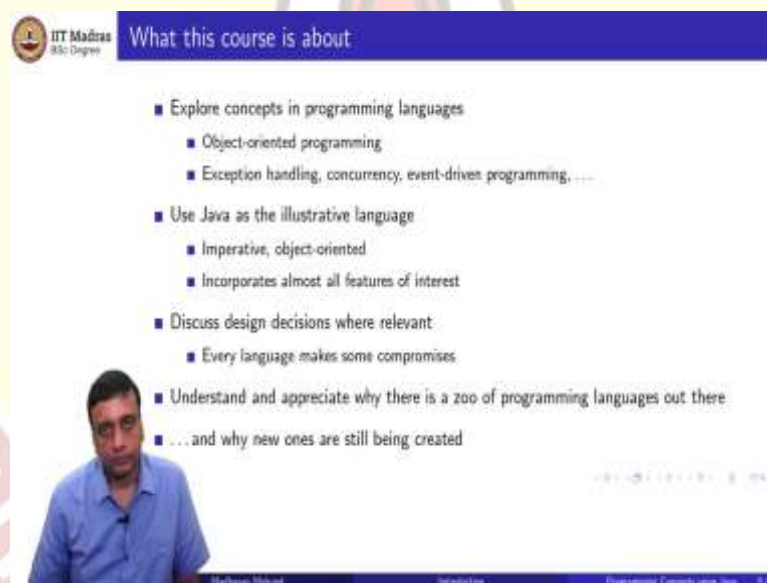
We write sequences of statements we learn if statements loops and all that right and data is only something which is passed around in between the statements we write functions pass data and all that. In an object-oriented approach we start with what are the data types that we need to manipulate and then we define the functions which are appropriate

for that. So, we kind of reverse the priority by looking at data as a central idea and functions are associated with the data on which they operate.

So, just as a concrete example in python we have for instance in the list data type we have this implicit function called sort which we can invoke on a particular list by having this dot notation versus the function sorted which takes a list from outside and passes you back a list. So, the second one sorted is a more conventional function it takes input data produces output data whereas the sort function is attached to a given list.

So, every list in a theoretical sense has its own sort function and it applies to that list. So, you say tell a list go and sort your data that is more or less how you tell the list to get itself sorted rather than taking a list and passing it to sort function. So, this kind of illustrates the difference between the two styles and we will explore it in more detail.

(Refer Slide Time: 22:46)



What this course is about

- Explore concepts in programming languages
 - Object-oriented programming
 - Exception handling, concurrency, event-driven programming, ...
- Use Java as the illustrative language
 - Imperative, object-oriented
 - Incorporates almost all features of interest
- Discuss design decisions where relevant
 - Every language makes some compromises
- Understand and appreciate why there is a zoo of programming languages out there
- ... and why new ones are still being created

So, with that background we can kind of get some idea of what this course is going to be about. So, basically we are going to look at some concepts and programming languages with a particular focus on object oriented programming. We will also on the way look at things like exception handling how do you deal with errors gracefully concurrency. If you have multiple things running at the same time how do you make sure that they do not interfere with each other.

Event driven programming this is what you use when you have program applications which look for mouse clicks and other kind of user inputs and decide what to do next.

Now for all this we will be studying these things in the context of java. So, java will be our base language with which we will illustrate these concepts. So, java itself is an imperative language it is a statically type language you have to declare your variables.

It is object oriented by design and it incorporates most of the features that we want to elaborate in a typical programming language concepts course. Now this does not mean that java on its own is perfect no language is perfect. So, we will also examine some of the decisions which maybe are not so useful or not well executed in java and why they are done because every language makes some compromises.

So, at the end of this I hope we will all understand better why there are so many languages right. Just as we have a natural language we have so, many languages spoken around the world we still have a zoo of programming languages in use some of them are historical right because we have been using languages like Fortran and Cobol and so on first for decades and it is very hard to change all of them to a new language.

But we still have new languages being created. So, if there are new languages being created clearly there is some deficiency in the languages that exist and clearly there is no one way of doing it. So, hopefully you will get an appreciation of what goes into designing a language and what are the features that are useful in some concept context and not in other contexts.